

Types and Declarations

We have not talked about the `void`, `short int`, and `long double` types. `void` is a type with no values, used as a placeholder to indicate functions that do not return values or that accept no arguments, and in the "generic" pointer type `void *` that can point to anything. `short int` is an integer type that might use less space than a plain `int`; `long double` is a floating-point type that might have even more range or precision than plain `double`.

The `char` type and the various sizes of `int` also have "unsigned" versions, which are declared using the keyword `unsigned`. Unsigned types cannot hold negative values but have guaranteed properties on overflow. (Whether a plain `char` is signed or unsigned is implementation-defined; you can use the keyword `signed` to force a character type to contain signed characters.) Unsigned types are also useful when manipulating individual bits and bytes, when "sign extension" might otherwise be a problem.

Two additional *type qualifiers* `const` and `volatile` allow you to declare variables (or pointers to data) which you promise not to change, or which might change in unexpected ways behind the program's back.

There are user-defined *structure* and *union* types. A structure or `struct` is a "record" consisting of one or more values of one or more types concreted together into one entity which can be manipulated as a whole. A `union` is a type which, at any one time, can hold a value from one of a specified set of types.

There are user-defined *enumeration* types ("enum") which are like integers but which always contain values from some fixed, predefined set, and for which the values are referred to by name instead of by number.

Pointers can point to functions as well as to data types.

Types can be arbitrarily complicated, when you start using multiple levels of pointers, arrays, functions, structures, and/or unions. Eventually, it's important to understand the concept of a *declarator*: in the declaration

```
int i, *ip, *fpi();
```

we have the *base type* `int` and three declarators `i`, `*ip`, and `*fpi()`. The declarator gives the name of a variable (or function) and also indicates whether it is a simple variable or a pointer, array, function, or some more elaborate combination (array of pointers, function returning pointer, etc.). In the example, `i` is declared to be a plain `int`, `ip` is declared to be a pointer to `int`, and `fpi` is declared to be a function returning pointer to `int`. (Complicated declarators may also contain parentheses for grouping, since there's a precedence hierarchy in declarators as well as expressions: `[]` for arrays and `()` for functions have higher precedence than `*` for pointers.)

We have not said much about pointers to pointers, or arrays of arrays (i.e. multidimensional arrays), or the ramifications of array/pointer equivalence on multidimensional arrays. (In particular, a reference to an array of arrays does *not* generate a pointer to a pointer; it generates a pointer to an array. You cannot pass a multidimensional array to a function which accepts pointers to pointers.)

Variables can be declared with a hint that they be placed in high-speed CPU registers, for efficiency. (These hints are rarely needed or used today, because modern compilers do a good job of register allocation by themselves, without hints.)

A mechanism called `typedef` allows you to define user-defined aliases (i.e. new and perhaps more-convenient names) for other types.

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995-1997 // [mail](#) [feedback](#)